

OBJECTS, INHERITANCE, AND LINKED LISTS Solutions

COMPUTER SCIENCE MENTORS 61A

October 21 – October 24, 2025

1 Inheritance and Representation

1. **Flying the cOOP** What would Python display? Write the result of executing the code and the prompts below. If a function is returned, write "Function". If nothing is returned, write "Nothing". If an error occurs, write "Error".
Hint: You may find it helpful to make an environment diagram tracking your objects and *each* new instance of an object (whether it's assigned to a variable or not!).

```
>>> andre.speak(Bird("coo"))

cluck
coo

class Bird:
    def __init__(self, call):
        self.call = call
        self.can_fly = True
    def fly(self):
        if self.can_fly:
            return "Don't stop me now!"
        else:
            return "Ground control to Major Tom..."
    def speak(self):
        print(self.call)

class Chicken(Bird):
    def speak(self, other):
        Bird.speak(self)
        other.speak()

class Penguin(Bird):
    can_fly = False
    def speak(self):
        call = "Ice to see you"
        print(call)

andre = Chicken("cluck")
gunter = Penguin("noot")

>>> andre.speak(gunter)

cluck
Ice to see you

>>> Bird.speak(gunter)

noot
```

Explanations

- `andre.speak(Bird("coo"))`

The `Bird` object is created and immediately passed in as the parameter for `Bird`. Even though we don't assign it to a variable, the object still exists and has all the features of a `Bird` object.

- `andre.speak()`

Python expects two parameters but in this case we are only assigning `self`.

- `gunter.fly()`

Note that the `Penguin` class will use the constructor for the `Bird` class, which sets `gunter.can_fly` for the particular instance.

- `andre.speak(gunter)`

This question is really similar to the first one, but instead of `Bird("coo")` we use the `gunter` object instead.

- `Bird.speak(gunter)`

`Bird.speak` looks within the `Bird` class to find the `speak` method.

2. What would Python display? Write the result of executing the following code and prompts. If nothing would happen, write "Nothing". If an error occurs, write "Error".

```
class ForceWielder():
    force = 25

    def __init__(self, name):
        self.name = name

    def train(self, other):
        other.force += self.force / 5

    def __str__(self):
        return self.name

class Jedi(ForceWielder):
    lightsaber = "blue"

    def __str__(self):
        return "Jedi " + self.name

    def __repr__(self):
        return f"Jedi({repr(self.name)}) "

class Sith(ForceWielder):
    lightsaber = "red"
    num_sith = 0

    def __init__(self, name):
        super().__init__(name)
        Sith.num_sith += 1
        if self.num_sith != 2:
            print("Two there should be. No more, no less.")

    def __str__(self):
        return "Darth " + self.name

    def __repr__(self):
        return f"Sith({repr(self.name)}) "
```

```

>>> anakin = Jedi("Anakin")
>>> anakin.lightsaber, anakin.force

("blue", 25)

>>> obiwan = Jedi("Obi-wan")
>>> anakin.master = obiwan
>>> anakin.master

Jedi("Obi-wan")

>>> Jedi.master

AttributeError

>>> obiwan.force += anakin.force
>>> obiwan.force, anakin.force

(50, 25)

>>> obiwan.train(anakin)
>>> obiwan.force, anakin.force

(50, 35.0)

>>> Jedi.train(obiwan, anakin)
>>> obiwan.force, anakin.force

(50, 45.0)

>>> sidious = Sith("Sidious")

Two there should be. No more, no less.

>>> ForceWielder.train(sidious, anakin)
>>> anakin.lightsaber = "red"
>>> anakin.lightsaber, anakin.force

("red", 50.0)

>>> Jedi.lightsaber

```

```
"blue"
```

```
>>> print(Sith("Vader"), Sith("Maul").num_sith)
```

```
Two there should be. No more, no less.  
Darth Vader 3
```

```
>>> rey = ForceWielder("Rey")  
>>> rey
```

```
<__main__.ForceWielder object>
```

```
>>> rey.lightsaber
```

```
AttributeError
```

2 Linked Lists

1. What will Python output? Draw box-and-pointer diagrams along the way.

```
>>> a = Link(1, Link(2, Link(3)))
```

```
+---+---+ +---+---+ +---+---+
| 1 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
1
```

```
>>> a.first = 5
```

```
+---+---+ +---+---+ +---+---+
| 5 | --|->| 2 | --|->| 3 | / |
+---+---+ +---+---+ +---+---+
```

```
>>> a.first
```

```
5
```

```
>>> a.rest.first
```

```
2
```

```
>>> a.rest.rest.rest.rest.first
```

```
Error: tuple object has no attribute rest (Link.empty has no rest)
```

```
>>> a.rest.rest.rest = a
```

```
      +---+---+ +---+---+ +---+---+
+-->| 5 | -->| 2 | -->| 3 | -->|
| +---+---+ +---+---+ +---+---+ |
|                                     |
+-----+-----+
```

```
>>> a.rest.rest.rest.rest.first
```

```
2
```

```
>>> repr(Link(1, Link(2, Link(3, Link.empty))))
```

```
"Link(1, Link(2, Link(3)))"
```

```
>>> Link(1, Link(2, Link(3, Link.empty)))
```

```
Link(1, Link(2, Link(3)))
```

```
>>> str(Link(1, Link(2, Link(3))))
```

```
'<1 2 3>'
```

```
>>> print(Link(Link(1), Link(2, Link(3))))
```

```
<<1> 2 3>
```


2. Write a function `reverse`, which takes in a `Link` and returns a new `Link` that has the order of the contents reversed.

Hint: You may want to use a helper function if you're solving this recursively.

```
def reverse(lst):
    """
    >>> a = Link(1, Link(2, Link(3)))
    >>> b = reverse(a)
    >>> b
    Link(3, Link(2, Link(1)))
    >>> a
    Link(1, Link(2, Link(3)))
    """
```

There are quite a few different methods. We have listed some here – can you think of any others?

```
# Recursive w/ Helper
def reverse(lst):
    def helper(so_far, rest):
        if rest is Link.empty:
            return so_far
        else:
            return helper(Link(rest.first, so_far), rest.rest)
    return helper(Link.empty, lst)

# Iterative
def reverse(lst):
    rev = Link.empty
    while lst is not Link.empty:
        rev = Link(lst.first, rev)
        lst = lst.rest
    return rev
```

3. Write a recursive function `insert_all` that takes as input two linked lists, `s` and `x`, and an index `index`. `insert_all` should return a new linked list with the contents of `x` inserted at index `index` of `s`.

```
def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
    Link(1, Link(1, Link(2, Link(5))))
    """
    if _____ and _____:
        _____

    if _____ and _____:
        _____

    _____

def insert_all(s, x, index):
    """
    >>> insert = Link(3, Link(4))
    >>> original = Link(1, Link(2, Link(5)))
    >>> insert_all(original, insert, 2)
    Link(1, Link(2, Link(3, Link(4, Link(5))))
    >>> start = Link(1)
    >>> insert_all(original, start, 0)
    Link(1, Link(1, Link(2, Link(5))))
    >>> insert_all(original, insert, 3)
    Link(1, Link(2, Link(5, Link(3, Link(4))))
    """
    if s is Link.empty and x is Link.empty:
        return Link.empty
    if x is not Link.empty and index == 0:
        return Link(x.first, insert_all(s, x.rest, 0))
    return Link(s.first, insert_all(s.rest, x, index - 1))
```

All of our return statements should return a new linked list.

Our base case should be the simplest possible version of the problem: when both `x` and `s` are empty, clearly the result is just the empty list.

We can now think of ways to break down this problem even further. Note that when the index to be inserted at is 0, the problem is relatively easy: we just have to put all of the elements of `x` followed by all the elements of `s`. So the first element of the new list should be `x.first`, and the rest of the new list should be `x.rest` concatenated with `s`, or `insert_all(s, x.rest, 0)`. Since we are using `x.first` and `x.rest`, we must check that `x` is nonempty to ensure that we do not error.

Finally, when the index to be inserted at is nonzero, we know that we're going to have some elements of `s`, then the elements of `x`, and then the rest of the elements from `s`. So the first element of the new list should be `s.first`. Then we can get the rest of the new list by inserting the contents of `x` at index `index - 1` of `s.rest`, reducing the index by 1 to account for the fact that we have removed the first element of `s`.

There's one issue we glossed over here: what if `x` is empty but `s` is not? Then we want to return the contents of `s`. But because the problem requires that we return a new linked list, we must recursively reconstruct `s` instead of simply returning it. You could add another base case to handle this, but as it turns out the second recursive case will handle this just fine since `Link(s.first, insert_all(s.rest, x, index - 1))` is just equivalent to `Link(s.first, s.rest)` when `x` is empty. Since the `x is not Link.empty` condition for the first recursive case will direct all situations where `x` is empty but `s` is not to the second recursive case, it turns out that we do not need to add anything else to this solution.

Convincing yourself that this problem works requires that you eventually reach a base case. Note that in either recursive call, we either reduce `s` or `x` by one element. So the base case will always eventually be reached, and the solution is valid.